



— SECURITY REVIEW · PREPARED FOR

Robin *Markets*

DATE

02.06.2026

CONDUCTED BY

Pyro, Lead Security Researcher
MarketerKA, Security Researcher
YanecaB, Security Researcher
Bobai23, Security Researcher

COMMIT

47d6b4a

REMEDIATION

663332f

Table of Contents

About Phage Security	3
Disclaimer	3
System overview	3
Overall Security Posture	3
Before remediation	3
After remediation	4
Technical Overview	4
Robin Markets Contracts (Polygon / Solidity 0.8.31)	4
Executive summary	5
Overview	5
Timeline	5
Scope	6
Findings Summary	7
Findings	8
High Severity	8
[H-01] POLY_GNOSIS_SAFE Signed Withdrawal Auth Doesn't Validate Current Safe Owner	8
[H-02] supplyIdleToVaults can be reentered through the 1155 hook to drain the protocol fee reserve	10
Medium Severity	11
[M-01] A misbehaving external vault can DOS deposits, withdrawals, and recovery	11
[M-02] Operator Pays Unbounded Gas When Signed-Withdrawal Recipient Burns Gas	12
[M-03] invalidateNonces Cannot Cancel Signed Withdrawals For Proxy / Safe Users	14
[M-04] Push-Deposit Inflates maximumAdditionalMatchedTokens	16
[M-05] Non-TWAP markets can be permanently blocked from finalization	18
Low Severity	19
[L-01] Pool loss is counted twice when one market side is empty	19
[L-02] nonZeroLength Is User-supplied and never validated against actual Non-Zero count	20
[L-03] setTwapOracle does not migrate per-market state and bricks every initialized market	22
[L-04] Incorrect return value in supplyIdleToVaults	23

About Phage Security

Phage Security is a team of highly skilled smart contract security researchers collaborating with 10+ proven freelancers who have excelled in public contests and private audits alike. With numerous vulnerabilities uncovered and patched across our completed audits, we strive to deliver the absolute best security service and client experience. While 100% security can never be guaranteed, we are committed to giving our utmost effort to protect your protocol.

Check out our previous work at PhageSecurity.com or reach out on X to [Pyro](#).

Disclaimer

Audits are a time, resource, and expertise-bound effort where trained experts evaluate smart contracts using a combination of automated and manual techniques to identify as many vulnerabilities as possible. Audits can reveal the presence of vulnerabilities **but cannot guarantee their absence**.

System overview

Robin lets users deposit Polymarket binary outcome tokens into a single vault and earn yield on the matched principal. Each deposit pairs YES with NO 1:1, merges the pair into USDC.e via Polymarket's Conditional Tokens Framework, and routes the USDC.e to external ERC-4626 vaults. Users get principal back as outcome tokens (reduced on loss), yield in USDC.e, and the protocol takes a basis-point cut of yield only.

Overall Security Posture

The audit found a system with sound foundations, with the deepest issues sitting in the signed-withdrawal authorization model and the seams where multi-market accounting meets the Polymarket and ERC-4626 boundaries.

Before remediation

Severity distribution: 2 High, 5 Medium, 4 Low. Share math, UUPS upgrade gating, and the dual-index loss/yield model held up. Issues clustered in signed-withdrawal authorization, cross-market accounting, TWAP-oracle coherence, and external vault resilience.

- **Signed-withdrawal authorization and operator path** (H-01, M-02, M-03) — `POLY_GNOSIS_SAFE` auth trusted the deploy-time EOA forever via a pure `CREATE2` derivation and never read the Safe's current owner set. `invalidateNonces` was keyed by `msg.sender` while `executeSignedWithdrawal` keyed by `withdrawal.user`, so cancels from proxy and Safe users silently no-opped, and the operator-paid path forwarded all gas to `withdrawal.user.onERC1155BatchReceived` with no stipend. In practice a single leaked or rotated-out deployer key kept permanent authority to drain a Safe user's yield and force-exit their position, the rightful owner had no on-chain way to cancel, and a hostile recipient could exhaust the operator's gas.
- **Cross-market accounting integrity** (H-02, M-04, L-01, L-02) — the permissionless `supplyIdleToVaults` was reachable through the ERC-1155 hook fired inside `_giveOutcomeTokens` and could be reentered to drain in-flight yield from the protocol's fee reserve. `_prepareDeposit`

`MintShares` double-counted the push-deposit balance, letting any user freeze new deposits vault-wide until an admin disables the capacity check; `IndexCalcLib` double-debited loss when one market side was empty, so depositors absorbed twice the realized loss on that market; and `nonZeroLength` was caller-supplied without verification against the actual non-zero entry count.

- **TWAP-oracle coherence** (M-05, L-03) — `submitTwap` was permissionless on non-required markets and could pin `lastTwapUpdate` past `marketEndedAt` to block finalization forever, stranding that market's principal and yield. `setTwapOracle` rewrote the oracle pointer with no per-market state migration, bricking every initialized market with no in-protocol recovery path.
- **External vault resilience** (M-01, L-04) — bare `IERC4626` calls in `_getVaultValue`, `_getAvailableCapacity`, `_withdrawFromVaults`, `_withdrawAllFromVault`, and `_withdrawMaxFromVault` meant one misbehaving registered vault could DoS `batchDeposit`, `batchWithdraw`, `harvestProtocolFee`, and the only recovery path. `supplyIdleToVaults` also returned the unsupplied remainder under a `supplied` variable name, reporting the exact opposite of reality.

After remediation

Fixed 9 of 11. Both High findings are fixed: the `POLY_GNOSIS_SAFE` branch now requires `ISafe(user).getThreshold() == 1 && ISafe(user).isOwner(signer)`, so a signed withdrawal is only valid for a current single-owner Safe (H-01), and `supplyIdleToVaults` was made `nonReentrant`, closing the 1155-hook reentrancy into the fee reserve (H-02). M-01 is partial — try/catch was added around `previewRedeem`, `maxDeposit`, `maxWithdraw`, and `withdraw`, but `_withdrawAllFromVault.redeem` stays bare and `_withdrawFromVaults` still does not skip `emergencyActivated` vaults. M-02 was acknowledged. Deployable once the operator gas-burn path (M-02) and the remaining bare ERC-4626 sites (M-01) are mitigated.

Technical Overview

Robin Markets Contracts (Polygon / Solidity 0.8.31)

Five in-scope contracts. `RobinStakingVault` and `RobinTwapOracle` are UUPS upgradeable behind `TIMELOCKED_ROLE`. `RobinStakingVaultExtension` is `DELEGATECALLED` from the vault's `fallback()`. `RobinLens` and `RobinTimeLockController` are not upgradeable.

- **RobinStakingVault** (`RobinStakingVault.sol`) — main UUPS singleton handling batch deposits, withdrawals, and signed withdrawals.
- **RobinStakingVaultExtension** (`RobinStakingVaultExtension.sol`) — admin surface reached via the vault's fallback, split across `TIMELOCKED_ROLE`, `DEFAULT_MANAGER_ROLE`, `OPERATOR_ROLE`, `PAUSER_ROLE`, and `FEE_HARVESTER_ROLE`.
- **RobinTwapOracle** (`RobinTwapOracle.sol`) — standalone UUPS oracle that ingests signed batch TWAP submissions via `submitTwap` and stores per-market accumulators and finalization data.
- **RobinLens** (`RobinLens.sol`) — stateless aggregator with `immutable vault` providing batch view helpers and `checkBatchDepositCapacity`.
- **RobinTimeLockController** (`RobinTimeLockController.sol`) — OpenZeppelin `TimeLockController` wrapper with `address(0)` as admin so no separate timelock admin exists.

Seven privileged roles operate the protocol:

- **DEFAULT_ADMIN_ROLE** — grants and revokes every non-timelocked role, held by the owner multisig.
- **DEFAULT_MANAGER_ROLE** — vault cap/active/remove/swap, Polymarket oracle list, URI, and `setTwapSigner` on the oracle, held by the owner multisig.
- **OPERATOR_ROLE** — emergency mode, per-vault emergency, `setTwapGracePeriod` (bounded by `MAX_TWAP_GRACE_PERIOD = 240`), and oracle TWAP-required flags, held by the multisig plus a manager EOA.
- **PAUSER_ROLE** — `setPauseAll` / `setPauseDeposits` / `setPauseWithdrawals` / `setPauseTransfers` on the vault and `pause/unpause` on the oracle, held by the owner multisig.
- **FEE_HARVESTER_ROLE** — calls `harvestProtocolFee(to)`, held by a dedicated harvester address.
- **TIMELOCKED_ROLE** — self-administered, gates `addVault`, `setProtocolFeeBps`, `setPolymarketOnramp`, `setPolymarketOfframp`, `setExtensionAddress`, and `_authorizeUpgrade`, held exclusively by `RobinTimeLockController`.
- **VAULT_ROLE** (oracle) — calls `initializeMarket` on the oracle when the vault first sees a new market, held by `RobinStakingVault`.

Executive summary

Overview

Project Name	Robin Markets
Repository	https://github.com/robin-markets/staking-vault-contracts
Commit hash	47d6b4a93d14c91c643e4032993ecaffa7e4b7e2
Remediation	663332f3c07a56fccb75e81e0fe2ac0b917bb16
Methods	Manual review

Timeline

Audit kick-off	18.05.2026
End of audit	22.05.2026
Remediations start	25.05.2026
Remediations end	27.05.2026

Scope

src/RobinLens.sol
src/RobinStakingVault.sol
src/RobinStakingVaultExtension.sol
src/RobinTimeLockController.sol
src/RobinTwapOracle.sol
src/libraries/AccountingLib.sol
src/libraries/IndexCalcLib.sol
src/libraries/PolymarketLib.sol
src/libraries/ShareMath.sol
src/libraries/StorageLib.sol
src/libraries/TwapMath.sol
src/libraries/VaultLib.sol
src/mixins/AccountingMixin.sol
src/mixins/PausableMixin.sol
src/mixins/PolymarketMixin.sol
src/mixins/SignaturesMixin.sol
src/mixins/YieldStrategyMixin.sol

Findings Summary



● 2 High · ● 5 Medium · ● 4 Low

ID	Title	Severity	Status
[H-01]	POLY_GNOSIS_SAFE Signed Withdrawal Auth Doesn't Validate Current Safe Owner	High	Fixed
[H-02]	<code>supplyIdleToVaults</code> can be reentered through the 1155 hook to drain the protocol fee reserve	High	Fixed
[M-01]	A misbehaving external vault can DOS deposits, withdrawals, and recovery	Medium	Fixed
[M-02]	Operator Pays Unbounded Gas When Signed-Withdrawal Recipient Burns Gas	Medium	Acknowledged
[M-03]	<code>invalidateNonces</code> Cannot Cancel Signed Withdrawals For Proxy / Safe Users	Medium	Fixed
[M-04]	Push-Deposit Inflates <code>maximumAdditionalMatchedTokens</code>	Medium	Fixed
[M-05]	Non-TWAP markets can be permanently blocked from finalization	Medium	Fixed
[L-01]	Pool loss is counted twice when one market side is empty	Low	Fixed
[L-02]	'nonZeroLength' Is User-supplied and never validated against actual Non-Zero count	Low	Fixed
[L-03]	'setTwapOracle' does not migrate per-market state and bricks every initialized market	Low	Fixed
[L-04]	Incorrect return value in <code>supplyIdleToVaults</code>	Low	Fixed

Findings

High Severity

[H-01] POLY_GNOSIS_SAFE Signed Withdrawal Auth Doesn't Validate Current Safe Owner

For `POLY_GNOSIS_SAFE`, Robin authorizes a signed withdrawal with a single check: `withdrawal.user == getSafeAddress(signer)`. That helper is a pure `CREATE2` derivation from `keccak256(signer)` — it never reads the Safe's current owner set. Robin keeps trusting the **deploy-time** EOA forever, even after the user removes that EOA from the Safe via `swapOwner`. A leaked or burner deployer key remains a permanent withdrawal authority, and the attacker can drain USDC yield directly to their own address by setting `yieldRecipient` in the signed message.

Root cause: Robin inherited the CTF Exchange `getSafeAddress(signer)` check, which is safe for a trade-based system but not for a withdrawal-based system with a user-controlled recipient.

The same `getSafeAddress(signer)` check exists in Polymarket's CTF Exchange — see `verifyPolySafeSignature`. There it is acceptable because a CTF Exchange order is an **atomic trade**: proceeds are hardcoded back to `order.maker` (the Safe), not to an attacker-chosen address — see `Trading._fillOrder(_transfer(msg.sender, order.maker, takerAssetId, taking - fee))`. The worst a leaked deploy-time key can do is force an unfavorable trade — funds stay in the Safe.

Robin reuses the check verbatim but adds an attacker-controlled field:

```
// DataTypes.sol:148
address yieldRecipient; // signer chooses where USDC yield goes
```

`yieldRecipient` is part of the EIP-712 message; whoever signs picks the address. So the same authorization that grants “permission to trade” in CTF becomes “permission to send USDC anywhere” in Robin. The pattern was inherited; the threat model wasn't. Polymarket itself moved past this design in their April 2026 V2 upgrade, which added `POLY_1271` as the recommended path for smart-contract-backed wallets — exactly because EIP-1271 honors the Safe's *current* signing rules instead of a frozen deploy-time derivation.

```
// SignaturesMixin.sol:159-170
} else if (signatureType == DataTypes.SignatureType.POLY_GNOSIS_SAFE) {
    return user == $.polymarketFactoryHelper.getSafeAddress(signer);
}
```

`getSafeAddress` is `CREATE2(salt = keccak256(signer))`. No call into the Safe, no `isOwner`, no `getThreshold`, no EIP-1271. The nonce bitmap is keyed on `user`, not `signer`, so the user has no per-signer revocation — they cannot retire one signing key without consuming every nonce on the Safe.

Example

Bob deployed his Safe a year ago with `0xOLD` (a Privy / relay / burner — the standard Polymarket onboarding pattern). He now uses a hardware wallet `0xNEW` and rotates ownership:

```
Safe.execTransaction(swapOwner(SENTINEL, 0xOLD, 0xNEW))→
Safe.isOwner(0xOLD) == false→
Safe.getOwners()    == [0xNEW]
```


`0x0LD` no longer controls the Safe via `Safe.execTransaction`. Bob assumes the key is retired.

Later, `0x0LD` leaks (old device backup, phishing capture made before rotation, lost laptop). The attacker signs a Robin signed withdrawal:

```
signer      = 0x0LD
user        = getSafeAddress(0x0LD) // Bob's Safe
signatureType = POLY_GNOSIS_SAFE
yieldRecipient = attacker_address ← stolen USDC ends up here
minYesTokens = 0 minNoTokens = 0 protectAgainstLoss = false
```

Robin checks `getSafeAddress(0x0LD) == user` → passes. Outcome tokens land in the Safe; USDC yield is sent to the attacker. Bob's position is force-exited at any price the attacker chooses.

Attack — multi-sig hardening (secondary)

Same primitive when the user adds a second owner with threshold 2 (`addOwnerWithThreshold(0xC0, 2)`) instead of removing the deployer. After this:

```
Safe.getOwners() == [0x0LD, 0xC0]
Safe.getThreshold() == 2
```

A single-sig `Safe.execTransaction` reverts with `GS020` (signatures data too short for threshold 2). But Robin still accepts a single signature from `0x0LD` because `getSafeAddress(0x0LD)` is unchanged.

In on-chain reality this variant is rare: top Polymarket users observed today are **1-of-1** Safes or `POLY_PROXY` clones (most top-50 traders use `POLY_PROXY`, which is not affected by this finding). Multi-sig hardening is currently a path “advanced users could take to defend themselves” rather than a default. The rotation variant is the one that fires routinely.

Code References

- **Only `POLY_GNOSIS_SAFE` auth** — `src/mixins/SignaturesMixin.sol#L159-L170`
- **Verify-and-consume** — `src/mixins/SignaturesMixin.sol#L122-L150`
- **Attacker-controlled `yieldRecipient`** — `src/types/DataTypes.sol#L143-L150`
- **SignatureType enum** — `src/types/DataTypes.sol#L128-L135`

IMPACT

- **Direct yield theft.** The attacker writes their own address into `yieldRecipient`; every accumulated yield epoch on Bob's position is divertable on demand. Outcome tokens still land in the Safe (they're hardcoded), but USDC walks out the door.
- **Forced exit.** With `minYesTokens = minNoTokens = 0`, the attacker liquidates Bob's position at any market price.
- **No on-chain revocation.** The nonce bitmap is keyed on `user` (the Safe), not `signer` (the leaked EOA). To revoke `0x0LD`, Bob would have to consume every nonce on the Safe and re-sign every legitimate order — impractical. There is no per-signer kill switch.
- **Asymmetry — new owner cannot help.** Even though `0xNEW` (or the multi-sig threshold) controls the Safe on the Safe side, `0xNEW`'s Robin signature for the existing Safe is **rejected** (`getSafeAddress(0xNEW)` is a different address). The legitimate owner cannot signed-withdraw or cancel from Robin without going through the Safe, while the leaked deployer can. The bug literally inverts the Safe ownership model.

The only mitigation a user has today is to fully withdraw from Robin and stop using it until a fix ships.

RECOMMENDATION

Replace the static derivation with EIP-1271 delegation against the Safe. EIP-1271 honors the Safe's current owner set and threshold (single-owner, multi-sig, or any future smart-wallet config), and it's the same direction Polymarket itself moved to — [POLY_1271](#) was added to the [CTF Exchange in V2](#) (April 2026):

```

} else if (signatureType == DataTypes.SignatureType.POLY_GNOSIS_SAFE) {
-   return user == $.polymarketFactoryHelper.getSafeAddress(signer);
+   return IERC1271(user).isValidSignature(digest, withdrawal.signature) ==
      0x1626ba7e;
}

```

This requires threading `digest` and the raw `signature` into `_verifySignerForUser` and removing the upstream `ECDSA.recover(...) != withdrawal.signer` check for the Safe branch (the Safe handles validation end-to-end).

If EIP-1271 is too invasive, the minimum patch is `ISafe(user).isOwner(signer)` — but it does **not** close the multi-sig variant (a still-present deployer in a 2-of-N Safe still passes with one signature). EIP-1271 closes both rotation and multi-sig with a single change.

References

- Polymarket CTF Exchange — `verifyPolySafeSignature` (same `getSafeAddress(signer)` check, acceptable there): github.com/Polymarket/ctf-exchange/blob/main/src/exchange/mixins/Signatures.s.sol
- Polymarket CTF Exchange — `Trading._fillOrder` (proceeds hardcoded to `order.maker`, no `yieldRecipient` equivalent): github.com/Polymarket/ctf-exchange/blob/main/src/exchange/mixins/Trading.s.sol
- Polymarket Signature Types overview (EOA, `POLY_PROXY`, `POLY_GNOSIS_SAFE`, `POLY_1271`): polymarket-ctf-exchange.mintlify.app/concepts/signature-types
- Polymarket Proxy Factories source (Safe deploy salt = `keccak256(ownerEOA)`, factory `0xaaacFeEa03eb1561C4e67d661e40682Bd20E3541b`): github.com/Polymarket/proxy-factories
- Safe owner-management primitives used in the attack: `swapOwner`, `addOwnerWithThreshold`
- EIP-1271 (recommended fix path): eips.ethereum.org/EIPS/eip-1271

[H-02] `supplyIdleToVaults` can be reentered through the 1155 hook to drain the protocol fee reserve

`_batchWithdraw` transfers outcome tokens to the user before paying out the USDC yield:

```

// Batch transfer all tokens to user
_giveOutcomeTokens(vars.ids, vars.amts, user);

// Handle payout, including protocol fee
(uint256 yield, uint256 protocolFee) = _handleYieldPayout(user,
    yieldRecipient, vars.totalYield, wrapYieldToPolyUsd);

```

`_giveOutcomeTokens` calls the CTF's `safeBatchTransferFrom`, which per ERC-1155 spec invokes the recipient's `onERC1155BatchReceived` hook after the balance update.

At this point the vault holds the in-flight yield as idle USDC alongside the `accumulatedProtocolFees` reserve, because `_ensureUsdcBalance` already pulled `splitNeeded + yield` from Yearn and `_split` only consumed `splitNeeded`.

`RobinStakingVaultExtension` has `supplyIdleToVaults` that can deposit funds to all listed vaults. The call is permissionless and reachable through the vault's `fallback`, which `DELEGATECALLs` into the extension.

```
function supplyIdleToVaults() external returns (uint256 supplied) {
    return _trySupplyIdleToVaults();
}
```

Inside the hook an attacker can call `vault.supplyIdleToVaults()._trySupplyIdleToVaults(reservedUsdc = accumulatedProtocolFees)` computes `idle = balance - reservedUsdc = yield` and sweeps that yield back into Yearn. Vault USDC balance drops to `accumulatedProtocolFees`.

Control returns to `_handleYieldPayout`. It transfers `yield` to the user out of what is now the fee reserve and bumps `accumulatedProtocolFees += protocolFee`. The user is paid the same amount they would have been paid in the legitimate flow, but the cash comes from the fee reserve and the real yield ends up as untracked principal sitting in Yearn.

This will also mess up internal accounting.

RECOMMENDATION

The cleanest fix is to add `nonReentrant` to `supplyIdleToVaults`:

```
- function supplyIdleToVaults() external returns (uint256 supplied) {
+ function supplyIdleToVaults() external nonReentrant returns (
  uint256 supplied) {
    return _trySupplyIdleToVaults();
  }
```

Medium Severity

[M-01] A misbehaving external vault can DOS deposits, withdrawals, and recovery

We integrate registered ERC-4626 vaults across `VaultLib`. Most external calls into them are bare (no `try/catch`). A single registered vault that reverts on any of these calls bricks the user facing path that reaches it.

The exposed sites and what they brick:

- `_getVaultValue` (`balanceOf`, `previewRedeem`) is reached via `_getTotalVaultValue`, which iterates every registered vault without skipping flagged ones. `_getTotalUsdcValue` calls it as the first external operation in `batchDeposit` and `batchWithdraw`. One reverting `previewRedeem` freezes 100% of TVL

- `_withdrawFromVaults` (`IERC4626.withdraw`) has no flag skip and no `try/catch`. An `emergencyActivated` vault that still reports `maxWithdraw > 0` and reverts on `withdraw` DOSes every withdrawal path and `harvestProtocolFee`
- `_trySupplyToVaults` (`IERC4626.deposit`) is documented “non-reverting” but its deposit call is bare
- `_getAvailableCapacity` (`maxDeposit`), `_withdrawAllFromVault` (`redeem`), and `_withdrawMaxFromVault` (`maxWithdraw`, `withdraw`) are all bare
- `enableVaultEmergency` and `enableEmergencyMode` call `_withdrawMaxFromVault` before setting the emergency flag, so a reverting vault leaves the flag unset and the bad vault eligible for future iteration. Recovery falls back to `removeVault` (timelocked), which itself calls the bare `_withdrawAllFromVault.redeem` and can revert too, leaving UUPS upgrade as the only escape

```
function _getVaultValue(address vault) private view returns (uint256) {
    uint256 shares = IERC4626(vault).balanceOf(address(this));
    return IERC4626(vault).previewRedeem(shares);
}

function _getTotalVaultValue() private view returns (uint256 total) {
    StorageLib.YieldStrategyStorage storage $ = _getStorage();
    for (uint256 i = 0; i < $.vaults.length; i++) {
        total += _getVaultValue($.vaults[i].vault);
    }
}
```

RECOMMENDATION

Wrap every external ERC-4626 call in `VaultLib` in `try/catch`. Example:

```
for (uint256 i = $.vaults.length; i > 0 && remaining > 0; i--) {
    DataTypes.ExternalVault storage v = $.vaults[i - 1];
    uint256 maxWithdraw = IERC4626(v.vault).maxWithdraw(address(this));
    if (maxWithdraw == 0) continue;
    uint256 toWithdraw = remaining < maxWithdraw ? remaining : maxWithdraw;
    - uint256 shares = IERC4626(v.vault).withdraw(toWithdraw, address(this),
    address(this));
    - emit IRobinStakingVaultEvents.VaultWithdrawal(v.vault, shares,
    toWithdraw);
    - remaining -= toWithdraw;
    + try IERC4626(v.vault).withdraw(toWithdraw, address(this), address(
    this)) returns (uint256 shares) {
    +     emit IRobinStakingVaultEvents.VaultWithdrawal(v.vault, shares,
    toWithdraw);
    +     remaining -= toWithdraw;
    + } catch {
    +     continue;
    + }
}
```

[M-02] Operator Pays Unbounded Gas When Signed-Withdrawal Recipient Burns Gas

`executeSignedWithdrawal` is operator-paid. It ends with `ctf.safeBatchTransferFrom(vault, withdrawal.user, ...)`, which invokes `withdrawal.user.onERC1155BatchReceived` with all forwarded gas (no stipend, no `try/catch`). An attacker can deploy code at `withdrawal.user` whose hook returns

the correct selector but burns ~5M gas first, draining the operator's hot wallet at zero cost to themselves.

```
// src/libraries/PolymarketLib.sol:102-110
function giveOutcomeTokens(uint256[] memory ids, uint256[] memory amts,
    address to) external {
    ctf.safeBatchTransferFrom(address(this), to, ids, amts, '');
}
```

```
// src/RobinStakingVault.sol:319-322
_giveOutcomeTokens(vars.ids, vars.amts, user); // <- forwards all gas to
    user.onERC1155BatchReceived
(uint256 yield, ) = _handleYieldPayout(user, yieldRecipient, vars.totalYield,
    wrapYieldToPolyUsd);
```

`_verifySignerForUser` (`src/mixins/SignaturesMixin.sol`) ties `signer` to `user` but does not constrain the code deployed at `user`.

- **EOA + EIP-7702 — exploitable.** `_verifySignerForUser` for EOA checks only `user == signer`, no `code.length` guard. A 7702-delegated EOA executes its delegate's code on the receive hook. (EIP-7702, live on Polygon since Pectra.)
- **POLY_GNOSIS_SAFE — exploitable.** `SafeProxyFactory.createProxy` is permissionless; any EOA can deploy its own Safe at the deterministic address Robin checks via `getSafeAddress(signer)`. As sole owner, the attacker calls `setFallbackHandler` to point at a gas-burning handler. ERC-1155 transfers to a Safe (no native receiver) fall through to `fallback`.
- **POLY_PROXY — not exploitable.** `ProxyWallet.onERC1155BatchReceived` is a pure function returning the selector. Wallets are EIP-1167 clones with the implementation baked into runtime bytecode; owners only have `proxy(calls)` and cannot replace the receive hook. (`ProxyWallet.sol`)

Example

1. Attacker EOA signs `CreateProxy` and calls `SafeProxyFactory.createProxy` (permissionless) — Safe deploys at `getSafeAddress(attackerEOA)` with attacker as sole owner.
2. Attacker `execTransaction` → `setFallbackHandler(MaliciousHandler)`. Handler returns `0xbc197c81` but burns ~5M gas first.
3. Attacker deposits, then signs `SignedWithdrawal { signatureType: POLY_GNOSIS_SAFE, user: attackerSafe, ... }`.
4. Backend submits → `safeBatchTransferFrom` → Safe `fallback` → `MaliciousHandler` → operator pays the burn.
5. USDC yield (line 322) goes through ERC-20 `transfer` after the burn and lands fine, so the attacker also collects their normal payout. Pure asymmetric griefing.

The EIP-7702 variant is analogous (delegate the EOA, sign with EOA).

IMPACT

At 50 gwei on Polygon, ~\$0.10 per malicious withdrawal. One funded EOA can pre-sign thousands of nonces and drain the operator's hot wallet, taking the limit-order backend offline. Attacker cost per attack: zero. Recovery requires off-chain mitigation (gas simulation before submit) or this fix.

RECOMMENDATION

Switch the operator-paid path to a pull pattern. Credit `claimableOutcomeTokens[user][id]` instead of pushing CTF, and let users claim on their own gas:

```
mapping(address => mapping(uint256 => uint256)) public claimableOutcomeTokens;

function _executeSignedWithdrawalInternal(SignedWithdrawal calldata sw)
    internal {
    ...
    for (uint i = 0; i < ids.length; i++) {
        claimableOutcomeTokens[sw.user][ids[i]] += amts[i];
    }
    emit OutcomeTokensCredited(sw.user, ids, amts);
}

function claimOutcomeTokens(uint256[] calldata ids) external nonReentrant {
    uint256[] memory amts = new uint256[](ids.length);
    for (uint i = 0; i < ids.length; i++) {
        amts[i] = claimableOutcomeTokens[msg.sender][ids[i]];
        claimableOutcomeTokens[msg.sender][ids[i]] = 0;
    }
    PolymarketLib.giveOutcomeTokens(ids, amts, msg.sender);
}
```

User-paid `batchWithdraw` can keep the push pattern (self-grief is moot).

If a pull pattern is too invasive, cap the gas forwarded to the recipient via assembly `call` with a fixed limit (e.g. 200K) and revert on overrun. Code-length checks on `user` don't help: `setFallbackHandler` can be re-set after a clean check (TOCTOU).

[M-03] `invalidateNonces` Cannot Cancel Signed Withdrawals For Proxy / Safe Users

Introduced in: `c991fcf` (move nonce invalidation to extension)

Cancel and execute use **different keys** to access the same nonce bitmap. For `POLY_PROXY` and `POLY_GNOSIS_SAFE`, a user calling `invalidateNonces` from their EOA writes a different storage slot than the one the executor reads — the cancel is a silent no-op and the withdrawal still executes. Raw EOA withdrawals (`user == signer`) are unaffected.

The mismatch

```
// RobinStakingVaultExtension.sol:244-247 - cancel keys by msg.sender
function invalidateNonces(uint256[] calldata nonces) external {
    _invalidateNonces(msg.sender, nonces);
    emit IRobinStakingVaultEvents.NoncesInvalidated(msg.sender, nonces);
}
```

```
// SignaturesMixin.sol:134-148 - execute keys by withdrawal.user
(bool isUsed, , ) = _getNonceInformation(withdrawal.user, withdrawal.nonce);
if (isUsed) revert WithdrawalNonceUsed(...);
$.nonceBitmap[withdrawal.user][wordPos] |= bit;
```


For `POLY_PROXY` / `POLY_GNOSIS_SAFE`, `withdrawal.user` is the proxy / Safe address — not the EOA:

```
// SignaturesMixin.sol:159-170
} else if (signatureType == DataTypes.SignatureType.POLY_PROXY) {
    return user == $.polymarketFactoryHelper.getPolyProxyWalletAddress(signer);
} else if (signatureType == DataTypes.SignatureType.POLY_GNOSIS_SAFE) {
    return user == $.polymarketFactoryHelper.getSafeAddress(signer);
}
```

`invalidateNonceWord` (line 250-253) has the same root cause — mass-cancel of 256 nonces is also a no-op for proxy / Safe users.

Example

Bob's MetaMask EOA: `0xAAAA`. His Polymarket proxy wallet: `0xPROXY`.

```
1. Bob signs: signer=0xAAAA, user=0xPROXY, nonce=7, signatureType=POLY_PROXY

2. Bob cancels from MetaMask: vault.invalidateNonces([7])
  → nonceBitmap[0xAAAA][0] gets the bit. // EOA slot
  → nonceBitmap[0xPROXY][0] still 0. // proxy slot untouched.
  Tx succeeds. NoncesInvalidated(0xAAAA, [7]) emitted. UI shows "cancelled".

3. Backend executes:
  reads nonceBitmap[0xPROXY][0] = 0 → isUsed=false → withdrawal goes
  through.
```

Code References

- **Cancel writes wrong key** — [src/RobinStakingVaultExtension.sol#L244-L247](#)
- **Cancel-word writes wrong key** — [src/RobinStakingVaultExtension.sol#L250-L253](#)
- **Execute reads correct key** — [src/mixins/SignaturesMixin.sol#L134-L148](#)
- **Why `user != signer` for proxy / Safe** — [src/mixins/SignaturesMixin.sol#L159-L170](#)
- **SignatureType enum (public API)** — [src/types/DataTypes.sol#L128-L135](#)

IMPACT

`POLY_PROXY` and `POLY_GNOSIS_SAFE` are first-class signature types and the default Polymarket flow — most traders interact through proxy or Safe wallets, not raw EOAs. Cancellation is the user's only on-chain way to revoke a signed withdrawal before `expiry`. With cancel broken, the user has handed the operator an **irrevocable execution authorization** for the lifetime of the order.

Concrete loss scenarios:

- **Forced exit on a winning position.** Bob signs `withdraw 1,000 YES, minYesTokens=800` at `YES=$0.80`. Market rallies to `$0.95`; Bob cancels and decides to hold for resolution. Market retraces to `$0.85`; operator executes (passes the \$800 floor). Bob's intended hold value \$1,000 → realized \$850. **Direct loss: \$150.**
- **Mass-cancel no-op.** Alice calls `invalidateNonceWord(0)` to wipe 256 pre-signed orders (\$25K notional). Markets move adverse; operator pulls everything still on-chain valid. Loss $\approx 4\% \times \$25K = \$1,000$ after a "successful" mass-cancel.
- **Compromised operator key.** Attacker holds the full backlog of pre-signed withdrawals and pulls each at the worst tick. EOA cancels are silent no-ops, so users cannot defend until they find the non-standard workaround. **Every outstanding signed-withdrawal balance is at risk.**

- **Double-exposure from off-chain drift.** Carol cancels an old order, sees `NoncesInvalidated`, signs a new order with different sizing. Operator executes both. Carol takes **double the intended exposure** plus rebalancing cost.

RECOMMENDATION

Cancellation works only if routed through the wallet itself, so that `msg.sender == proxyOrSafe`:

- Safe: `Safe.execTransaction(vault, 0, invalidateNonces([n]), ...)` — owner-signed.
- Proxy: `proxyWallet.proxy([{ to: vault, data: invalidateNonces([n]) }])`.

Not the natural call surface, not what the public docstring “cancel up to 256 orders at once” suggests, not exposed through any standard wallet UX. A typical user pressing “cancel” in MetaMask hits the broken path.

To fix the issue take the wallet address and `signatureType` as parameters and reuse `_verifySignerForUser` to authorize the caller:

```
- function invalidateNonces(uint256[] calldata nonces) external {
-   _invalidateNonces(msg.sender, nonces);
-   emit NoncesInvalidated(msg.sender, nonces);
- }
+ function invalidateNonces(
+   address user,
+   DataTypes.SignatureType signatureType,
+   uint256[] calldata nonces
+ ) external {
+   if (!_verifySignerForUser(msg.sender, user, signatureType)) {
+     revert InvalidWithdrawalSignature();
+   }
+   _invalidateNonces(user, nonces);
+   emit NoncesInvalidated(user, nonces);
+ }
```

[M-04] Push-Deposit Inflates `maximumAdditionalMatchedTokens`

Introduced in: `3dbf5b2` — “support push deposits via onERC1155Received hooks for Polymarket DepositWallets”

The vault tracks a global counter `maximumAdditionalMatchedTokens` — how much USDC could potentially enter the yield pool if all unpaired tokens were matched. This counter is checked on every deposit: if it exceeds vault capacity, the deposit reverts.

The counter update logic in `_prepareDepositMintShares` was written for pull-deposits, where the vault reads its token balance **before** tokens arrive. For push-deposits the order is reversed — tokens land on the vault first (per ERC-1155 spec), then the code runs. The code still reads the balance and adds the deposit amount on top, counting the same tokens twice.

Each push-deposit that pairs with existing opposite-side tokens leaves a permanent phantom value in the counter. Repeated push-deposits accumulate this inflation until the counter exceeds capacity and **all deposits revert** — both push and pull, for all users.

`RobinStakingVault.sol:493-494`:


```
(uint256 oldYes, uint256 oldNo) = getUnpairedTokens(conditionId);
- _updateMaxPotential(oldYes, oldNo, oldYes + yesAmount, oldNo + noAmount);
+
+
+ BUG: in push path, oldYes ALREADY contains
  yesAmount
```

- **Pull path.** `getUnpairedTokens()` returns the balance **before** the deposit (correct), so `oldYes + yesAmount` equals the expected new balance.
- **Push path.** `getUnpairedTokens()` returns the balance **after** the deposit (tokens already landed), so `oldYes + yesAmount` equals 2× the deposit — double-counted.

ERC-1155 Spec Proof

From [Polymarket CTF ERC1155.sol:124-128](#):

```
_balances[id][to] = value.add(_balances[id][to]); // L124: balance
  updated FIRST

emit TransferSingle(msg.sender, from, to, id, value);

_doSafeTransferAcceptanceCheck(msg.sender, from, to, id, value,
  data); // L128: hook called AFTER
```

And from their own [IERC1155TokenReceiver.sol](#):

“called at the end of a `safeTransferFrom` after the balance has been updated”

So when Robin’s `onERC1155Received` fires and calls `ctf.balanceOf(address(this), positionId)`, the pushed tokens are already counted in the result.

IMPACT

Any user can permanently block all deposits vault-wide by repeatedly push-depositing into markets that have opposite-side unpaired tokens. No funds are lost — withdrawals and yield remain unaffected — but the vault stops accepting new deposits until an admin disables the capacity check.

RECOMMENDATION

In `_prepareDepositMintShares`, subtract the deposit amounts when in push path to reconstruct the true pre-transfer state:

```
- (uint256 oldYes, uint256 oldNo) = getUnpairedTokens(conditionId);
- _updateMaxPotential(oldYes, oldNo, oldYes + yesAmount, oldNo + noAmount);
+ (uint256 currentYes, uint256 currentNo) = getUnpairedTokens(conditionId);
+ if (tokensPrePushed) {
+   // Push path: tokens already in balance, reconstruct pre-transfer state
+   _updateMaxPotential(currentYes - yesAmount, currentNo - noAmount,
+     currentYes, currentNo);
+ } else {
+   // Pull path: tokens not yet here, current IS the old state
+   _updateMaxPotential(currentYes, currentNo, currentYes + yesAmount,
+     currentNo + noAmount);
+ }
```

This requires passing `tokensPrePushed` into `_prepareDepositMintShares` (it's already available in `_batchDepositCore`).

[M-05] Non-TWAP markets can be permanently blocked from finalization

`submitTwap` is permissionless and accepts unsigned data for markets that don't require TWAP. The branch in `_processTwapForMarket` for such markets routes through `_applyDefaultTwap`:

```
if (!isTwapRequired) {
  _applyDefaultTwap(conditionId);
  return false;
}
```

`_applyDefaultTwap` unconditionally sets `lastTwapUpdate = block.timestamp`:

```
function _applyDefaultTwap(bytes32 conditionId) internal {
  ...
  market.lastTwapUpdate = uint40(block.timestamp);
}
```

When the backend later submits the real finalization, `_applyFinalTwap` enforces:

```
if (twapData.marketEndedAt <= market.lastTwapUpdate) {
  revert TwapTimestampInvalid(market.lastTwapUpdate, twapData.marketEndedAt,
    market.lastTwapUpdate + 1);
}
if (twapData.marketEndedAt > block.timestamp) {
  revert TwapTimestampInvalid(market.lastTwapUpdate, twapData.marketEndedAt,
    block.timestamp);
}
```

`marketEndedAt` is the historical Polymarket resolution timestamp. It cannot be both `> lastTwapUpdate` (which any user can set to `block.timestamp` by calling `submitTwap`) and `<= block.timestamp`. Any user can call `submitTwap` every block to keep `lastTwapUpdate` pinned to `block.timestamp` and block finalization indefinitely.

Example:

1. Market resolves on Polymarket at `t = R`, YES wins (price `1e6`)
2. Attacker calls `submitTwap` with `required = false`, then `lastTwapUpdate = T`, `T > R`
3. Backend submits the finalization payload with `marketEndedAt = R`, but the TX reverts `TwapTimestampInvalid`
4. This can be repeated so finalization is permanently blocked

Even if not repeated every block (to DOS the market from finalization), the attack still allows yield to be split 50/50 after a market has been finalized

RECOMMENDATION

In `_applyFinalTwap` for `!twapRequired` markets, clamp instead of revert. The default-price extrapolation already covered the gap at the correct 50:50 weighting, so we can just skip the delta and snap

forward.

```
-     if (twapData.marketEndedAt <= market.lastTwapUpdate) {
-         revert TwapTimestampInvalid(market.lastTwapUpdate,
twapData.marketEndedAt, market.lastTwapUpdate + 1);
-     }

+     uint256 effectiveEnd = twapData.marketEndedAt > market.lastTwapUpdate
+         ? twapData.marketEndedAt
+         : market.lastTwapUpdate;

// ...
-     uint256 timeToEnd = twapData.marketEndedAt - market.lastTwapUpdate;
+     uint256 timeToEnd = effectiveEnd - market.lastTwapUpdate;
```

Low Severity

[L-01] Pool loss is counted twice when one market side is empty

When a market takes a loss, `IndexCalcLib.calculateIndexes` debits the same `delta` twice.

First the `LOSS` branch reduces both `lossIndexYes` and `lossIndexNo` by the full `delta`:

```
vars.yesDelta = vars.delta;
vars.noDelta = vars.delta;
```

Then `_applyExcessLossReduction` reduces `yieldReductionFactor` by the part of the loss past the pairable backing:

```
uint256 pairableTokens = yesTokenAssets < noTokenAssets ?
    yesTokenAssets : noTokenAssets;
if (delta <= pairableTokens) return;

uint256 excessUsdc = delta - pairableTokens;
```

Only the paired slice `min(YES, NO)` is backed by splittable pool USDC, where 1 USDC is 1 YES plus 1 NO. A loss inside that slice belongs on `lossIndex`. A loss past it cannot sit on `lossIndex`, because the surviving side is backed by unpaired outcome tokens, so it must hit yield instead.

Example:

1. Market has 100 YES shares, 0 NO shares, and 10 USDC of unclaimed YES yield in the pool
2. NO side is empty, so `pairableTokens = min(100, 0) = 0`
3. Pool loses 5 USDC, `delta = 5`
4. The `LOSS` branch debits `lossIndexYes` by 5
5. `_applyExcessLossReduction` sees `excessUsdc = 5 - 0 = 5` and debits `yieldReductionFactor` by 5

6. YES took a loss of 5 YES and yield is reduced by 5 USDC.

	Code does	Correct
YES backing (lossIndexYes)	100 -> 95	100 -> 100
yield (yieldReductionFactor)	10 -> 5	10 -> 5

RECOMMENDATION

Cap the side `lossIndex` reduction at `pairableTokens` instead of applying the full `delta`.

```

    } else {
-       vars.yesDelta = vars.delta;
-       vars.noDelta = vars.delta;

        uint256 yesTokenAssets =
            vars.totalSharesYes > 0 ? vars.totalSharesYes.mulDiv(
                result.lossIndexYes, DataTypes.INDEX_SCALE,
                Math.Rounding.Floor) : 0;
        uint256 noTokenAssets =
            vars.totalSharesNo > 0 ? vars.totalSharesNo.mulDiv(
                result.lossIndexNo, DataTypes.INDEX_SCALE,
                Math.Rounding.Floor) : 0;

+         uint256 pairableTokens = yesTokenAssets < noTokenAssets ?
yesTokenAssets : noTokenAssets;
+         uint256 lossToIndex = vars.delta < pairableTokens ? vars.delta :
pairableTokens;
+         vars.yesDelta = lossToIndex;
+         vars.noDelta = lossToIndex;

        if (vars.totalSharesYes > 0 && vars.yesDelta > 0) {
            vars.yesChange = vars.yesDelta.mulDiv(DataTypes.INDEX_SCALE,
                vars.totalSharesYes, Math.Rounding.Floor);
            result.lossIndexYes = result.lossIndexYes > vars.yesChange ?
                result.lossIndexYes - vars.yesChange : 0;
        }
        if (vars.totalSharesNo > 0 && vars.noDelta > 0) {
            vars.noChange = vars.noDelta.mulDiv(DataTypes.INDEX_SCALE,
                vars.totalSharesNo, Math.Rounding.Floor);
            result.lossIndexNo = result.lossIndexNo > vars.noChange ?
                result.lossIndexNo - vars.noChange : 0;
        }
    }

```

[L-02] nonZeroLength Is User-supplied and never validated against actual Non-Zero count

`nonZeroLength` is passed by the caller in `batchDeposit` and `batchWithdraw` but is only checked to be `>= vars.len`, never verified against the actual count of non-zero entries across the `yesAmounts/noAmounts` arrays:

```

if (nonZeroLength < vars.len) revert LengthMismatch();

```

This means a caller can pass any length when the real non-zero count is between `vars.len` to `vars.len*2`, causing tail entries remain zero-initialized (`ids[i] = 0`, `amts[i] = 0`) and are passed into

`_takeOutcomeTokens` / `_giveOutcomeTokens` as-is.

For the pull-deposit path this silently attempts to transfer zero amounts of token ID 0 to the vault — harmless today but produces malformed `TransferBatch` events that break off-chain indexers. For the push-deposit path, the `keccak256` of the inflated arrays will never match `receivedHash`, so it reverts with a misleading `PushDepositMismatch` instead of `LengthMismatch`, masking the real cause.

RECOMMENDATION

Count the actual non-zero entries before array allocation and require it equals `nonZeroLength`. Move the per-entry zero check into this pre-validation loop so the zero-amount guard and the count validation share a single pass:

```
function _batchDepositCore(
    address depositor,
    bytes32 receivedHash,
    bytes32[] memory conditionIds,
    bytes32[] memory questionIds,
    uint256[] memory yesAmounts,
    uint256[] memory noAmounts,
    uint256 nonZeroLength,
    uint256 referralCode,
    bool tokensPrePushed
) internal {
    DataTypes.BatchDepositVars memory vars;

    vars.len = conditionIds.length;
    if (vars.len == 0) revert ZeroAmount();
    if (yesAmounts.length != vars.len || noAmounts.length != vars.len ||
        questionIds.length != vars.len)
        revert LengthMismatch();

+ // Validate nonZeroLength matches the actual number of non-zero token
  entries.
+ uint256 actualNonZeroCount;
+ for (uint256 i = 0; i < vars.len; i++) {
+     if (yesAmounts[i] == 0 && noAmounts[i] == 0) revert ZeroAmount();
+     if (yesAmounts[i] > 0) actualNonZeroCount++;
+     if (noAmounts[i] > 0) actualNonZeroCount++;
+ }
+ if (nonZeroLength != actualNonZeroCount) revert LengthMismatch();

- if (nonZeroLength < vars.len) revert LengthMismatch();

    _updatePoolAssets(_getTotalUsdcValue());

    vars.ids = new uint256[](nonZeroLength);
    vars.amts = new uint256[](nonZeroLength);
    vars.yesShares = new uint256[](vars.len);
    vars.noShares = new uint256[](vars.len);

    vars.nonZeroIndex = 0;
    for (uint256 i = 0; i < vars.len; i++) {
        if (i > 0 && conditionIds[i] <= conditionIds[i - 1]) revert
            UnsortedConditionIds();
        uint256 yesAmount = yesAmounts[i];
        uint256 noAmount = noAmounts[i];
-     if (yesAmount == 0 && noAmount == 0) revert ZeroAmount(
    ); // now caught in pre-validation
```

```

    // ... rest unchanged
  }
}

```

Apply the same pattern in `_batchWithdraw`:

```

+ uint256 actualNonZeroCount;
+ for (uint256 i = 0; i < vars.len; i++) {
+   if (yesSharesArr[i] == 0 && noSharesArr[i] == 0) revert ZeroAmount();
+   if (yesSharesArr[i] > 0) actualNonZeroCount++;
+   if (noSharesArr[i] > 0) actualNonZeroCount++;
+ }
+ if (nonZeroLength != actualNonZeroCount) revert LengthMismatch();
- if (nonZeroLength < vars.len) revert LengthMismatch();

```

[L-03] `setTwapOracle` does not migrate per-market state and bricks every initialized market

`setTwapOracle` lives on the extension and is gated by `TIMELOCKED_ROLE`. The underlying mixin setter just overwrites the storage pointer:

```

function _setTwapOracle(address twapOracle_) internal {
  _getAccountingStorage().twapOracle = IRobinTwapOracle(twapOracle_);
}

```

No per-market state is copied to the new oracle. Every market that was initialized on the old oracle is uninitialized on the new one, while the vault's `MarketState.lastYieldTwapCheckpointYes` still carries the old oracle's accumulator value from the last index update.

After the swap, every deposit and withdraw path runs `_updateYieldIndexes`, which calls `AccountingLib.getTwapAccumulatorYes`:

```

function getTwapAccumulatorYes(bytes32 conditionId) public view returns (
  uint256 twapAccumulatorYes, uint256 lastUpdate) {
  StorageLib.AccountingStorage storage $ = _getAccountingStorage();
  (twapAccumulatorYes, lastUpdate) = $.twapOracle.getCurrentTwapAccumulator(
    conditionId);
  if (lastUpdate == 0) revert IRobinStakingVaultErrors.MarketNotInitialized(
    conditionId);
}

```

In the realistic migration scenario the new oracle is deployed with TWAP enabled (`globalTwapRequired = true`). `getCurrentTwapAccumulator` then hits the TWAP-required branch:

```

return (market.twapAccumulatorYes, market.lastTwapUpdate);

```

For a market that was never initialized on the new oracle this returns `(0, 0)`, the vault check reverts with `MarketNotInitialized`, and every deposit and withdraw on every active market is DoS'd.

Note that recovery after this change is also not possible (with the current setup).

The natural recovery path through the vault is also blocked. `vault.initializeMarket(conditionId, questionId)` ends in `AccountingLib.initializeMarket`:

```
if (market.marketInitTimestamp > 0) revert
  IRobinStakingVaultErrors.MarketAlreadyInitialized(conditionId);
```

The vault still has `marketInitTimestamp > 0` from the original initialization, so the recovery call reverts with `MarketAlreadyInitialized`. The only way to bootstrap the new oracle is to call `new Oracle.initializeMarket(...)` directly under `VAULT_ROLE`, bypassing the vault's accounting entry point.

Re-initializing on the new oracle does not fully recover the system. After re-init, the new oracle reports `lastTwapUpdate = T_reinit` and `twapAccumulatorYes = 0`. The vault check now passes, but execution continues into `IndexCalcLib.calculateIndexes`:

```
vars.twapAccumulatorYesDelta = input.twapAccumulatorYes -
  market.lastYieldTwapCheckpointYes;
```

`input.twapAccumulatorYes = 0` from the fresh oracle, while `market.lastYieldTwapCheckpointYes` is the non-zero value left over from the old oracle. The subtraction underflows and the call reverts with `Panic(0x11)`. Deposits and withdrawals are still DoS'd.

Calling `submitTwap` on the new oracle is not a workaround. The submit loop skips markets that are not initialized on that oracle:

```
for (uint256 i = 0; i < twapData.markets.length; i++) {
  if (!isMarketInitialized(twapData.markets[i].conditionId)) continue;
  ...
}
```

So before re-init `submitTwap` is a no-op. After re-init, `_applyTwap` only advances the accumulator from `lastTwapUpdate` forward and silently skips any `endTimestamp <= lastTwapUpdate`, so there is no way to backfill the historical accumulator to match the old vault checkpoint.

RECOMMENDATION

Remove the bare setter. Replace it with a migration function that takes the list of existing condition IDs, initializes each one on the new oracle, and resets the vault's per-market checkpoints in the same transaction.

[L-04] Incorrect return value in `supplyIdleToVaults`

The `supplyIdleToVaults` function in `RobinStakingVaultExtension` is designed to permissionlessly push idle USDC to external yield vaults. It returns a `supplied` variable intended to report exactly how much USDC was moved. However, the internal call to `_trySupplyIdleToVaults()` actually returns the remaining unsupplied) amount of idle USDC.

By passing this return value straight through, the function reports the exact opposite of reality (e.g., returning `0` when all funds were successfully supplied, and returning the full idle balance when the external vaults were full and `0` funds were supplied).


```
function supplyIdleToVaults() external returns (uint256 supplied) {  
    return _trySupplyIdleToVaults();  
}
```

```
function _trySupplyIdleToVaults() internal returns (uint256 remaining) {  
    return VaultLib.trySupplyIdleToVaults(_getReservedUsdc());  
}
```

```
function trySupplyIdleToVaults(uint256 reservedUsdc) external returns (  
    uint256 remaining) {  
    uint256 idle = _getIdleUsdc(reservedUsdc);  
    if (idle == 0) return 0;  
    remaining = _trySupplyToVaults(idle, reservedUsdc);  
}
```

RECOMMENDATION

Calculate the actual amount supplied by determining the idle balance before the internal call and subtracting the returned `remaining` balance.